

TEST DRIVEN DEVELOPMENT

En gestión tradicional el testing se deja para el final, pero esto genera problemas como el acoplamiento incidental. A la hora de diseñarlo no se divide el problema en partes pequeñas, por lo que la configuración de las pruebas y el entorno sea difícil. Ante el mínimo cambio, implicaba retrabajo y reingeniería para solucionar los problemas. El costo asintótico del error no es constante, sino que va incrementando. Era un enfoque reactivo.

Acoplamiento incidental – Deuda técnica y miedo al cambio – Costo asintótico del error.

Estos problemas dieron el paso al TDD. Plantea un practica que cambia el paradigma. TDD no es testing, es una técnica de desarrollo.

TDD (Kent Beck): es una práctica/actividad de especificación y diseño en microciclos basándose en pruebas unitarias. Por medio de pruebas definimos la meta que debe cumplir el software, para luego al cómo. Pasamos a un enfoque proactivo. Se define el éxito y se construye en base a él.

3 leyes de aplicación del TDD:

1. NO escribir código de producción, sin antes haber escrito un test unitario que falle. Implica una gran cobertura de pruebas.
2. NO vas a escribir más código de prueba del necesario para provocar un fallo. Si el código ni siquiera compila, ya es un fallo. Saber cuándo cortar
3. NO vas a escribir más código de producción que el mínimo indispensable para que la prueba pase.

ETAPAS:

1- RED: Pensar, DECLARAR lo que debe hacer. Trabajamos de manera declarativa. Declaramos la intención: “Quiero que, al pasar estos datos, el sistema devuelva este resultado”. Debe tener etapa de Precondiciones (crear entorno objeto = new Objeto()), Pasos (donde se ejecuta rtado= objeto.funcion(p1=n; p2=n), Resultado o Evaluacion (assert o expect(rtd == n))

```
def test_sumar_dos_numeros_EXITO():
    //Precondiciones
    calculadoraDePrueba = new Calculadora()
    //Pasos
    resultado = calculadoraDePrueba.sumar(a = 1, b = 2)
    //Resultado
    assert(resultado == 3)

Class Calculadora:
    def sumar(a,b):
        return "No implementado" //Puede ser pass
        directamente
```

2- GREEN: pasamos a un enfoque imperativo (escribir el cómo). Escribimos el código mínimo para que la prueba pase. Podemos usar el patron “Fake-it”, hardcodeando el resultado y disminuyendo la complejidad para no perder de vista las alternativas de los casos de prueba.

```
Class Calculadora:
    def sumar(a,b):
        return 3
```

3- REFACTOR: modificar la estructura interna, sin alterar el comportamiento externo.

```
Class Calculadora:
    def sumar(a: Number, b: Number) -> Number:
        return 3
```

El ciclo se arranca de nuevo para que el código interno se cambie para acercarme al objetivo planteado en el primer test. Por ejemplo, cambiar las entradas y ver como determinar las salidas en la etapa de green. Esto sirve para pasar de la particularización del problema a la generalización.

```
def test_sumar_dos_numeros_EXITO():
    //Precondiciones
    calculadoraDePrueba = new Calculadora()
    //Pasos
    resultado = calculadoraDePrueba.sumar(a = 3, b = 2)
    //Resultados
    assert(resultado == 5) //nueva comparación a + b

Class Calculadora:
    def sumar(a: Number, b: Number) -> Number:
        return a + b
```

Patrones y buenas prácticas:

RED: ayudan a definir que queremos hacer.

- Starter Test: dice que elijamos la prueba mas trivial y el camino más obvio para pivotear y complejizar más tarde, y hacer que el problema para que el flujo comience.
- One Step Test: seria crear la siguiente prueba más fácil.
- Explanantion test: nos permite debatir sobre el comportamiento esperado del sistema, se convierte ese debate en un test para poder solucionarlo. Sirve para definir el comportamiento del sistema. Para etapas maduras.
- Learning test: sirve para aprender a utilizar código externo (va de la mano del Explanation), APIs, librerías para luego integrarlo de manera efectiva. Tests para herramientas externas. Sirve para definir el comportamiento del sistema utilizando software externo.

- Another Test: sirve para ayudar a no perder el foco. Se debe anotar y poner como pendientes la lista de alternativas y seguir con el foco original.
- Regression Test: cuando ocurre un problema se debe definir un test para llegar a ese problema y replicar el error para que este no vuelva a suceder.

GREEN: hacer que la prueba pase de todas formas.

- Fake-it: hardcodear la prueba para obtener el resultado.
- Triangulación: armar varios casos de falla para asegurarme de que la falla está en la lógica de la aplicación. Llevar el problema particular a uno genérico para todas las alternativas.
- Obvious Implementation: si es algo muy obvio, se debe hacer la implementación correcta para cumplir con la prueba en todos los casos posibles. Funciona en problemas con complejidad sencilla.
- One to many: si tengo una función para colecciones, entonces hay que enfocarse en una unidad y en cada iteración se arma para toda la colección.

DISEÑO DE TEST :

- Test List: antes de desarrollar, se define un listado para todas las pruebas a hacer.
- Isolated Tests: es mas una característica, las pruebas deben ser lo más independientes posibles.
- Test First: va de la mano de la primera ley. Primero la prueba y luego el código.
- Assert Test: primero se escribe la evaluación y luego se dice que herramientas y entornos deben estar configurados.
- Test Data: si tengo un cálculo complejo, la relación entre Entradas y salidas debe ser lo más obvia posible.

Otros patrones:

- Testing
- Diseño
- Xunit Patterns

Ver mapa de taxis

5

Como Administrador de la Central quiero ver la ubicación de todos los taxis de la central y si tienen viajes en curso para saber la disponibilidad actual

Criterios de Aceptación:

- Se muestran taxis en todos los estados (Verde: Libre, Amarillo: Solicitado, Rojo: Ocupado, Negro: Fuera de Servicio).
- Los taxis que se mostrarán deben tener habilitado el sistema de ubicación.
- Pasando el mouse sobre el ícono del auto deben poder visualizarse los datos del viaje si el taxi está ocupado (hora de inicio y costo) y los datos del pasajero (nombre, apellido y número de teléfono celular).
- Si el taxi está solicitado, sólo se visualizarán los datos del pasajero.
- Podrán utilizarse filtros por Barrio, por Estado y por chapa del taxi

Pruebas de Usuario

- Probar visualizar un conjunto de taxis libres (pasa)
- Probar visualizar un conjunto de taxis solicitados (pasa)
- Probar visualizar un conjunto de taxis fuera de servicio (pasa)
- Probar visualizar un conjunto de taxis ocupados (pasa)
- Probar visualizar un conjunto de taxis libres, solicitados, fuera de servicio y ocupados (pasa)
- Probar visualizar un taxi conectado al sistema por número de chapa (pasa)
- Probar visualizar en el mapa un taxi no conectado al sistema de ubicación (falla)
- Probar visualizar un conjunto de taxis en un Barrio inexistente (falla)

**EJEMPLO FINAL**

```
def mostrar_mapa(filtros: Filtros, barrio: Barrio, chapa: String, estado: Estado) -> Mapa:
```

```
// Caso de prueba taxis desconectados
```

```
if (filtros == 'Mostrar todos los taxis')
  taxis = buscarTaxiEnBarrio(barrio)
  (ubicación == None) //ubicación desconectada
  taxis = buscarTaxiDesconectado (ubicacion)
```

```
// Caso de prueba mostrar taxis en distintos estados en un barrio
```

```
if (filtros == 'Mostrar taxis por estado y por barrio')
  taxis = buscarTaxiPorEstadoYPorBarrio (barrio, estado)
```

```
// Caso de prueba mostrar taxis por chapa
```

```
if (filtros == 'Mostrar taxis por chapa')
  taxis = buscarTaxiPorChapa (chapa)
```

```
// Caso de prueba mostrar taxis por barrio y datos de viaje
```

```
if (filtros == 'Mostrar taxis por barrio')
  taxisFiltrados = buscarTaxiEnBarrio (barrio)
  //foreach simplificado
  if (taxis[i].getEstado() == 'Solicitado' && filtros == 'Mostrar datos viaje')
    taxis =
      taxis[i].getChapa()
      taxis[i].getViajeActual().getPasajero().getNombre()
      taxis[i].getViajeActual().getPasajero().getTelefono()
      taxis[i].getViajeActual().getFechaYHora()
```

```
return new Mapa(taxis=taxis)
```

Pruebas de Usuario

- Probar visualizar un conjunto de taxis libres (pasa)
- Probar visualizar un conjunto de taxis solicitados (pasa)
- Probar visualizar un conjunto de taxis fuera de servicio (pasa)
- Probar visualizar un conjunto de taxis ocupados (pasa)
- Probar visualizar un conjunto de taxis libres, solicitados, fuera de servicio y ocupados (pasa)
- Probar visualizar un taxi conectado al sistema por número de chapa (pasa)
- Probar visualizar en el mapa un taxi no conectado al sistema de ubicación (falla)
- Probar visualizar un conjunto de taxis en un Barrio inexistente (falla)



RED

```
def test_mostrar_mapa_taxis_libres_en_barrio_exito():
    //Precondiciones
    usuarios = [{(user: "salvita", rol: "Administrador de la Central)]]
    loguear(salvita)
    configurar_ciudad_prueba("Córdoba Capital")
    taxis = [{(chapa: "ABC123", estado: "Libre", barrio: "Nueva Córdoba", ubicacion: (10, 20))}]

    //Pasos
    mapa_mostrado = mostrar_mapa(filtros = ["barrio", "estado"], barrio = "General Paz", chapa = "", estado = "Libre")
    //Resultado
    assert all((mapa_mostrado.ciudad.barrio == "General Paz", "No existe tal barrio para esa ciudad ingresada")
    assert all((taxi.barrio == "General Paz" for taxi in mapa_mostrado.taxis, "No todos los taxis están dentro del área filtrada")
    assert all((taxi.estado == "Libre" for taxi in mapa_mostrado.taxis, "No todos los taxis están libres")
    assert all((taxi.icono == "verde" for taxi in mapa_mostrado.taxis, "No todos los taxis tienen el ícono verde")

def mostrar_mapa(filtros, barrio, chapa, estado):
    pass
```



GREEN

```
def test_mostrar_mapa_taxis_libres_en_barrio_exito():
    //Precondiciones
    usuarios = [{(user: "salvita", rol: "Administrador de la Central)]]
    loguear(salvita)
    configurar_ciudad_prueba("Córdoba Capital")
    //carga los datos al sistema.
    taxis = [{(chapa: "ABC123", estado: "Libre", barrio: "General Paz", ubicacion: (10, 20))}]

    //Pasos
    mapa_mostrado = mostrar_mapa(filtros = ["barrio", "estado"], barrio = "General Paz", chapa = "", estado = "Libre")
    //Resultado
    assert all((mapa_mostrado.ciudad.barrio == "General Paz", "No existe tal barrio para esa ciudad ingresada")
    assert all((taxi.barrio == "General Paz" for taxi in mapa_mostrado.taxis, "No todos los taxis están dentro del área filtrada")
    assert all((taxi.estado.nombre == "Libre" for taxi in mapa_mostrado.taxis, "No todos los taxis están libres")
    assert all((taxi.estado.icono == "verde" for taxi in mapa_mostrado.taxis, "No todos los taxis tienen el ícono verde")

def mostrar_mapa(filtros, barrio, chapa, estado):
    return new Mapa([(taxi={chapa = "ABC123", estado = {nombre = "Libre", icono = "verde"}, barrio = "General Paz")}]
```



REFACTOR

```
def mostrar_mapa(filtros: String[], barrio: Barrio, chapa: String, estado: Estado) -> Mapa:
    estado_libre = new Estado(
        nombre = "Libre",
        icono = "verde"
    )

    barrio_general_paz = new Barrio(
        nombre = "General Paz"
    )

    taxi = new Taxi(
        chapa = "ABC123",
        estado = estado_libre,
        barrio = barrio_general_paz
    )

    taxis = [taxi]

    return new Mapa(taxis = taxis)
```

1. Volver a iniciar el ciclo.
2. Considerar el resto de casos de pruebas, añadiendo las alternativas planteadas en la user.
3. Refactorizar.

Hacer TDD sin refactorizar es simplemente escribir código malo con pruebas.